Milestone 3

Christopher Zion

Grand Canyon University

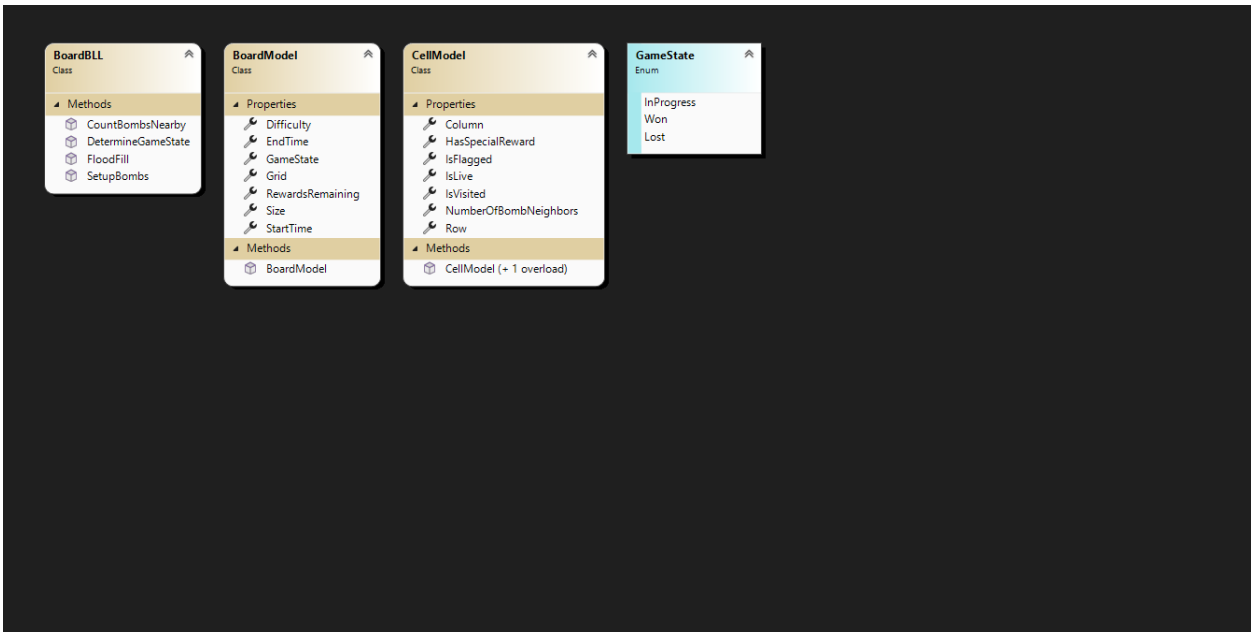
Course Number: CST-250

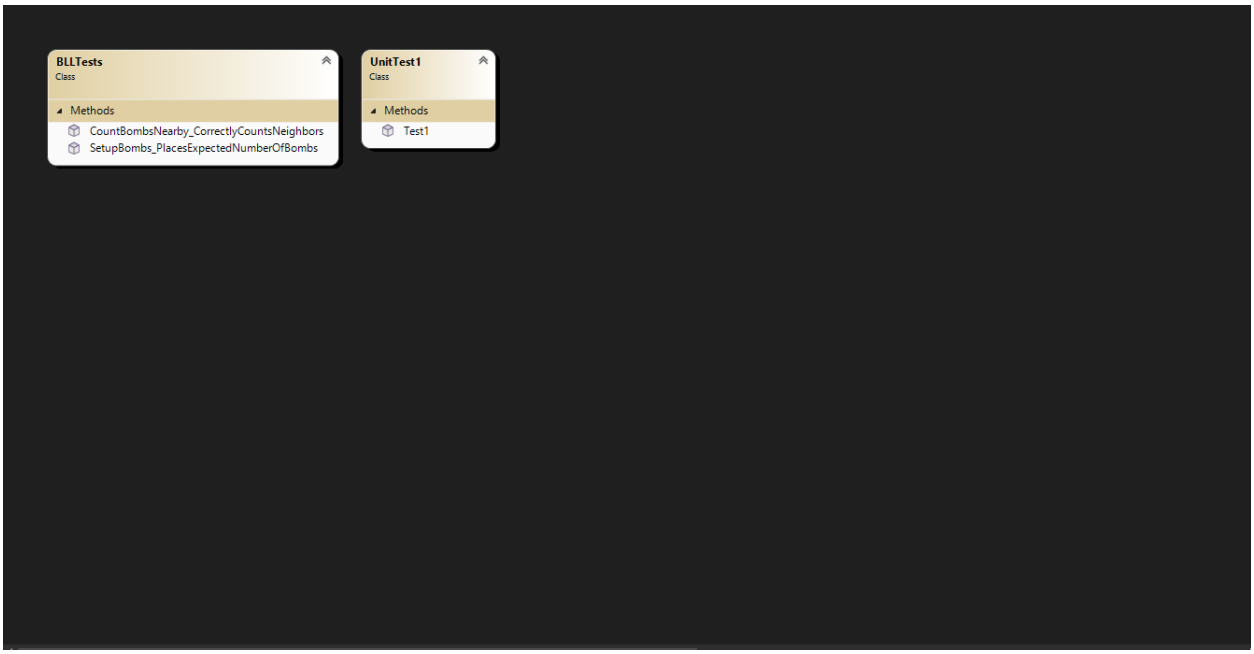
Github Repo:

https://github.com/cziongcu/Minesweeper-Milestone

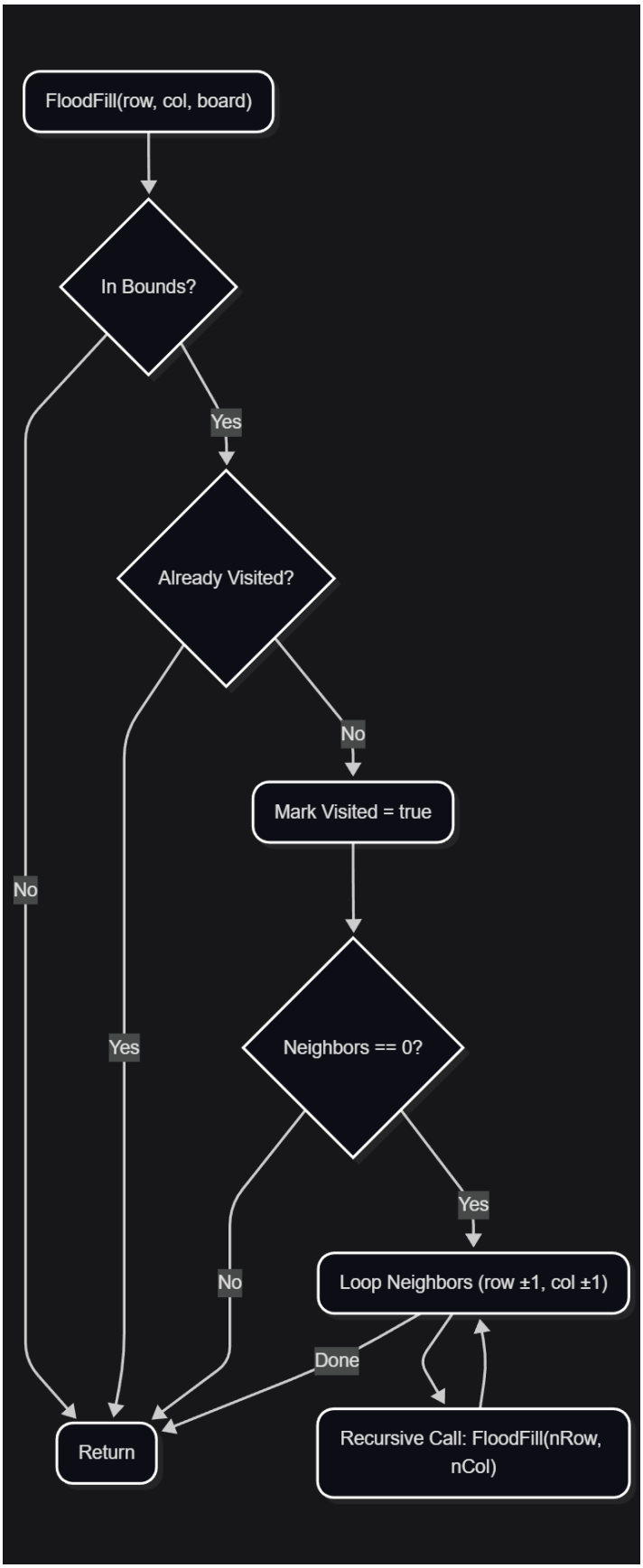
UML Diagrams:

Program
Class
Methods
Main
PrintAnswers
PrintBoard

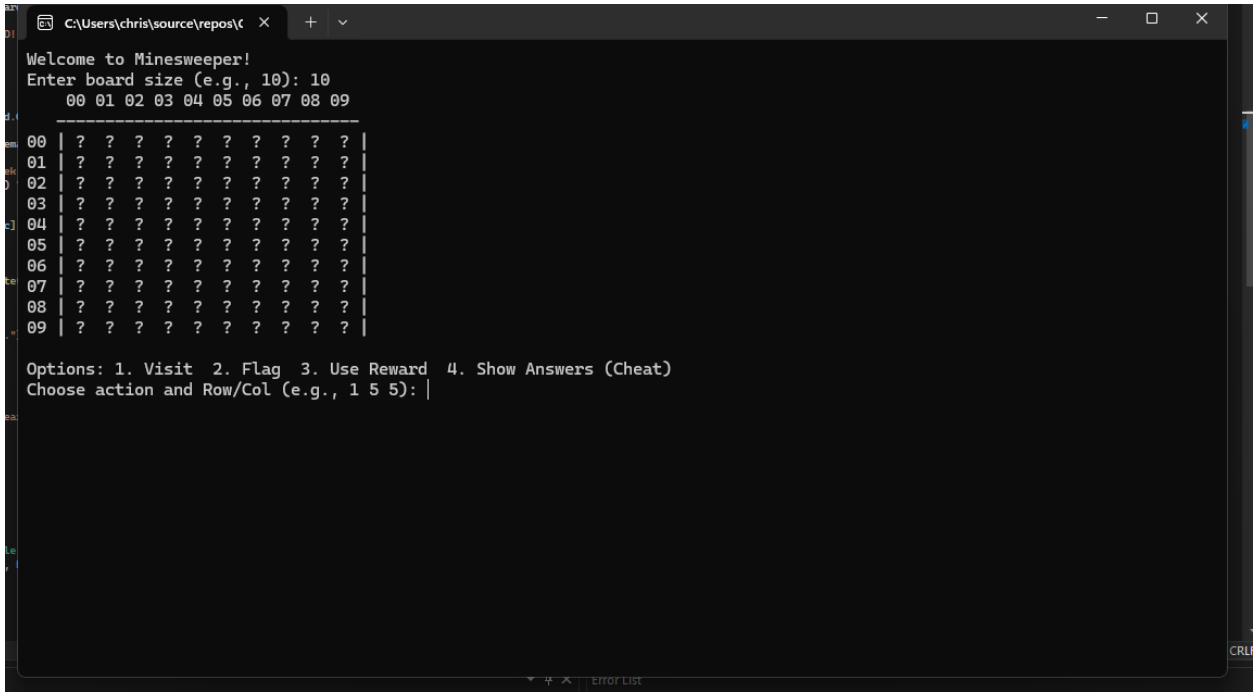

BoardBLL
Class
Methods
CountBombsNearby
DetermineGameState
FloodFill
SetupBombs
BoardModel
Class
Properties
Difficulty
EndTime
GameState
Grid
RewardsRemaining
Size
StartTime
Methods
BoardModel
CellModel
Class
Properties
Column
HasSpecialReward
IsFlagged
IsLive
IsVisited
NumberOfBombNeighbors
Row
Methods
CellModel (+ 1 overload)
GameState
Enum
InProgress
Won
Lost

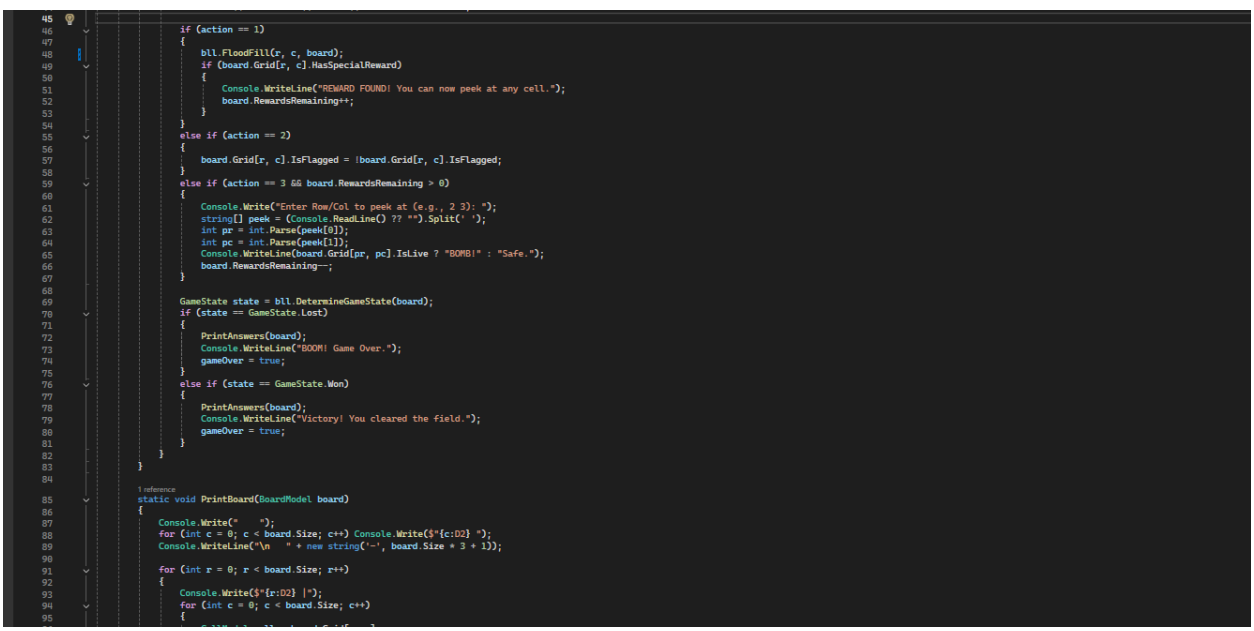

BLLTests
Class
Methods
CountBombsNearby_CorrectlyCountsNeighbors
SetupBombs_PlacesExpectedNumberOfBombs
UnitTest1
Class
Methods
Test1

Flowcharts:


FloodFill(row, col, board)
In Bounds?
Yes
No
Already Visited?
Yes
No
Mark Visited = true
Neighbors == 0?
Yes
No
Loop Neighbors (row ±1, col ±1)
Done
Return
Recursive Call: FloodFill(nRow, nCol)

Pictures:





```
            if (action == 1)
            {
                bll.FloodFill(r, c, board);
                if (board.Grid[r, c].HasSpecialReward)
                {
                    Console.WriteLine("REWARD FOUND! You can now peek at any cell.");
                    board.RewardsRemaining++;
                }
            }
            else if (action == 2)
            {
                board.Grid[r, c].IsFlagged = !board.Grid[r, c].IsFlagged;
            }
            else if (action == 3 && board.RewardsRemaining > 0)
            {
                Console.Write("Enter Row/Col to peek at (e.g., 2 3): ");
                string[] peek = (Console.ReadLine() ?? "").Split(' ');
                int pr = int.Parse(peek[0]);
                int pc = int.Parse(peek[1]);
                Console.WriteLine(board.Grid[pr, pc].IsLive ? "BOMB!" : "Safe.");
                board.RewardsRemaining--;
            }

            GameState state = bll.DetermineGameState(board);
            if (state == GameState.Lost)
            {
                PrintAnswers(board);
                Console.WriteLine("BOOM! Game Over.");
                gameOver = true;
            }
            else if (state == GameState.Won)
            {
                PrintAnswers(board);
                Console.WriteLine("Victory! You cleared the field.");
                gameOver = true;
            }
        }
    }

    1 reference
    static void PrintBoard(BoardModel board)
    {
        Console.Write("    ");
        for (int c = 0; c < board.Size; c++) Console.Write($"{c:D2} ");
        Console.WriteLine("\n   " + new string('-', board.Size * 3 + 1));

        for (int r = 0; r < board.Size; r++)
        {
            Console.Write($"{r:D2} |");
            for (int c = 0; c < board.Size; c++)
            {
```

```
{
    0 references
    static void Main(string[] args)
    {
        BoardBLL bll = new BoardBLL();
        Console.WriteLine("Welcome to Minesweeper!");
        Console.Write("Enter board size (e.g., 10): ");
        int size = int.Parse(Console.ReadLine() ?? "10");

        BoardModel board = new BoardModel(size);
        board.Difficulty = 15; // default 15%
        bll.SetupBombs(board);
        bll.CountBombsNearby(board);

        bool gameOver = false;
        while (!gameOver)
        {
            PrintBoard(board);
            Console.WriteLine("\nOptions: 1. Visit  2. Flag  3. Use Reward  4. Show Answers (Cheat)");
            Console.Write("Choose action and Row/Col (e.g., 1 5 5): ");
            string[] input = (Console.ReadLine() ?? "").Split(' ');

            if (input.Length < 3 && input[0] != "4") continue;

            int action = int.Parse(input[0]);
            if (action == 4) { PrintAnswers(board); continue; }

            int r = int.Parse(input[1]);
            int c = int.Parse(input[2]);

            if (r < 0 || r >= size || c < 0 || c >= size) continue;

            if (action == 1)
            {
                bll.FloodFill(r, c, board);
                if (board.Grid[r, c].HasSpecialReward)
                {
                    Console.WriteLine("REWARD FOUND! You can now peek at any cell.");
                    board.RewardsRemaining++;
                }
            }
            else if (action == 2)
            {
                board.Grid[r, c].IsFlagged = !board.Grid[r, c].IsFlagged;
            }
            else if (action == 3 && board.RewardsRemaining > 0)
            {
                Console.Write("Enter Row/Col to peek at (e.g., 2 3): ");
                string[] peek = (Console.ReadLine() ?? "").Split(' ');
                int pr = int.Parse(peek[0]);
                int pc = int.Parse(peek[1]);
                Console.WriteLine(board.Grid[pr, pc].IsLive ? "BOMB!" : "Safe.");
                board.RewardsRemaining--;
```

```
C:\Users\chris\source\repos\(

    00 01 02 03 04 05 06 07 08 09
  -------------------------------
00 | ?  ?  ?  ?  ?  ?  ?  ?  ?  ? |
01 | ?  ?  ?  ?  ?  ?  ?  ?  ?  ? |
02 | ?  ?  ?  ?  ?  ?  ?  ?  ?  ? |
03 | ?  ?  ?  ?  ?  ?  ?  ?  ?  ? |
04 | ?  ?  ?  ?  F  ?  ?  ?  ?  ? |
05 | ?  ?  ?  ?  ?  1  ?  ?  ?  ? |
06 | ?  ?  ?  ?  ?  ?  ?  ?  ?  ? |
07 | ?  ?  ?  ?  ?  ?  ?  ?  ?  ? |
08 | ?  ?  ?  ?  ?  ?  ?  ?  ?  ? |
09 | ?  ?  ?  ?  ?  ?  ?  ?  ?  ? |

Options: 1. Visit  2. Flag  3. Use Reward  4. Show Answers (Cheat)
Choose action and Row/Col (e.g., 1 5 5): 1 4 5
    00 01 02 03 04 05 06 07 08 09
  -------------------------------
00 | ?  ?  ?  ?  ?  ?  ?  ?  ?  ? |
01 | ?  ?  ?  ?  2  1  3  ?  ?  ? |
02 | ?  ?  ?  ?  1  .  1  ?  ?  ? |
03 | ?  ?  ?  ?  1  .  1  1  ?  ? |
04 | ?  ?  ?  ?  1  .  .  1  ?  ? |
05 | ?  ?  ?  ?  1  1  1  2  ?  ? |
06 | ?  ?  ?  ?  ?  ?  ?  ?  ?  ? |
07 | ?  ?  ?  ?  ?  ?  ?  ?  ?  ? |
08 | ?  ?  ?  ?  ?  ?  ?  ?  ?  ? |
09 | ?  ?  ?  ?  ?  ?  ?  ?  ?  ? |

Options: 1. Visit  2. Flag  3. Use Reward  4. Show Answers (Cheat)
Choose action and Row/Col (e.g., 1 5 5):
```

Summary:

In this milestone, I explored how Recursion works by implementing a Flood Fill algorithm to automatically clear out sections of the board. Basically, I wrote a method that calls itself to check and reveal neighboring cells, spreading out across the grid until it hits a stopping point. The most critical part was defining a Base Case, like hitting the edge of the board, a bomb-adjacent cell, or one we've already checked because without that stop condition, the code would just loop infinitely and crash the program.